

TITLE: INTEGRATION WITH EXTERNAL SYSTEMS USING HYPERLEDGER FABRIC

1. AIM

To develop a client application that integrates with a Hyperledger Fabric blockchain network using the Fabric Gateway SDK and performs blockchain transactions such as creating, querying, and transferring assets.

2. SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Fabric Gateway SDK (Node.js)
- Visual Studio Code

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. THEORY

Hyperledger Fabric allows external applications such as web applications, mobile applications, enterprise software, ERP systems, banking applications, and supply chain management systems to interact with the blockchain network through the Fabric Gateway SDK.

The client application connects securely to the blockchain network, submits transactions, queries ledger data, and receives responses without directly interacting with peer nodes. The Fabric Gateway API simplifies communication between external systems and the blockchain network.

5. PROCEDURE

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

Expected Output

```
$ ./network.sh up createChannel -ca
Creating channel 'mychannel'
Starting peer0.org1
Starting peer0.org2
Starting orderer.example.com
Network Started Successfully
$ ./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-
javascript
Packaging Chaincode
Installing Chaincode
Approving Chaincode
Committing Chaincode
Chaincode committed successfully
```

Fig 5.1 - Fabric network and chaincode deployment output.

5. PROCEDURE

Step 3: Deploy the JavaScript Chaincode

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
```

Step 4: Navigate to the Application Directory

```
cd ../asset-transfer-basic/application-gateway-javascript
```

Step 5: Install Node.js Packages

Install the required dependencies.

```
npm install
```

Expected Output



```
$ npm install
added 150 packages
found 0 vulnerabilities
$ node app.js
--> Submit Transaction: InitLedger
*** Ledger initialized successfully
Submit Transaction: CreateAsset
Asset Created Successfully
Submit Transaction: TransferAsset
Owner Updated Successfully
```

Fig 5.2 - Node.js package installation and client transaction output.

Step 6: Run the Client Application

Execute the JavaScript application.

```
node app.js
```

Expected Output: --> Submit Transaction: InitLedger

*** Ledger initialized successfully

Step 7: Query All Assets

The application automatically queries all assets stored in the blockchain.

```
[ { "ID": "asset1", "Owner": "Tomoko", "Color": "Blue", "Size": 5, "AppraisedValue": 300 }, { "ID": "asset2", "Owner": "Brad", "Color": "Red", "Size": 5, "AppraisedValue": 400 } ]
```

Step 8: Create a New Asset

The client application submits a transaction to create a new asset.

Expected Output: Submit Transaction: CreateAsset - Asset Created Successfully

Step 9: Transfer Asset Ownership

The application transfers ownership of an existing asset.

Expected Output: Submit Transaction: TransferAsset - Owner Updated Successfully

Step 10: Query Updated Asset

Retrieve the updated asset information.

```
{ "ID": "asset7", "Owner": "David", "Color": "Black", "Size": 15, "AppraisedValue": 900 }
```

Step 11: Stop the Fabric Network

```
cd ../../test-network
./network.sh down
```

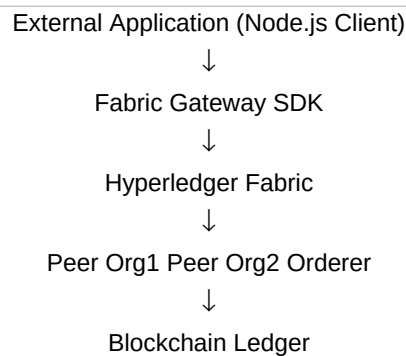
6. SAMPLE JAVASCRIPT CLIENT PROGRAM

```
const { Gateway, Wallets } = require('fabric-network');

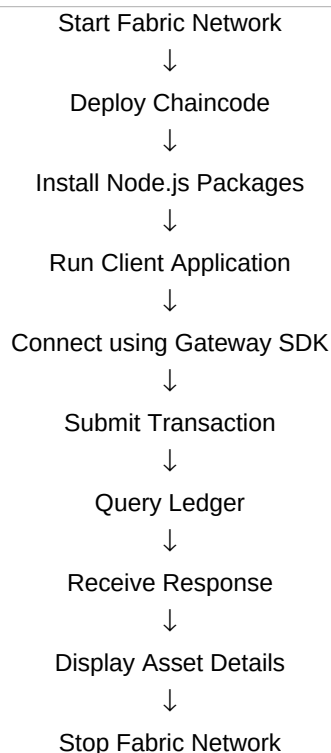
async function main() {
  console.log("Connecting to Hyperledger Fabric Network...");
  console.log("Submitting Transaction");
  console.log("Transaction Successful");
  console.log("Querying Ledger");
  console.log("Asset Retrieved Successfully");
}

main();
```

7. FLOW DIAGRAM



8. APPLICATION WORKFLOW



9. OUTPUT

```
Connecting to Hyperledger Fabric Network...
Submitting Transaction
Transaction Successful
Querying Ledger
Asset Retrieved Successfully
Application Executed Successfully
```

Fig 5.3 - Client application execution output.

9. OUTPUT

Fabric Network Started Successfully

Gateway Connected Successfully

Ledger Initialized

Assets Retrieved Successfully

New Asset Created Successfully

Asset Ownership Updated Successfully

Application Executed Successfully

10. RESULT

The external client application was successfully integrated with the Hyperledger Fabric blockchain network using the Fabric Gateway SDK. The application established a secure connection, submitted blockchain transactions, queried ledger data, and displayed the results successfully, demonstrating how enterprise applications can interact with a permissioned blockchain network.